

```

clear; clc;
load("droneArmMaterials.mat")
droneArmMaterials = materials;
materialTable = struct2table(droneArmMaterials);
% Display material data
disp(materialTable)

g = 9.81; % Gravitational Constant

Battery_kg=0.3;           % Battery mass, kg
PerMotor_kg=65/1000;      % Mass of each motor, kg
Total4Motor_kg=4*PerMotor_kg; % Total Mass of 4 motor, kg

PerPropeller_kg=10/1000;  % Mass of each propeller, kg
Total4Propeller_kg=4*PerPropeller_kg; % Total Mass of each propeller, kg

Peresc_kg=30/1000;        % Mass of each electronic speed controller, kg
Total4esc_kg=4*Peresc_kg;

electronicsMass_kg=0.05;  %Flight controller and electronics mass, kg
GPSSMass_kg=0.03;        % Receiver/GPS mass, kg
centerFrameMass_kg=0.2;   % Center frame mass excluding arms, kg

% units in kg
baseMass = Total4esc_kg + Total4Propeller_kg + Total4Motor_kg + electronicsMass_kg +
GPSSMass_kg+ centerFrameMass_kg+ Battery_kg; % Mass of entire drone excluding payload
and arms
numDesigns = 2;
numMaterials = 6;
minimumPayload = 0.5; % Minimum supported payload requirement
ratio = 2; % Required thrust to weight ratio 2:1
totalThrust_N= 4 * g; % 4 motors total, each with 1 kg thrust, converted to Newtons

% Drone materials in order:
materials = {droneArmMaterials.name}';

designs = {'Design 1';
'Design 2';
'Design 3'}

% Design 1,2,3 in order
armMass = [0.012, 0.021, 0.015, 0.0095, 0.0080, 0.0046; %Design 1 Mass of drone arm
0.071, 0.12, 0.084, 0.047, 0.047, 0.027; % Design 2 Mass of drone arm
0.036, 0.06, 0.042, 0.028, 0.023, 0.013]; % Design 3

```

```
maxPayload = zeros(numDesigns,numMaterials);
```

```
function results = thrustToWeightAnalysis(baseMass,armMass,totalThrust_N,designs,materials)
```

```
i = 1;
```

```
g = 9.81; % Gravitational Constant
```

```
numDesigns = size(armMass,1);
```

```
numMaterials = size(armMass,2);
```

```
minimumPayload = 0.5; % Minimum supported payload requirement
```

```
ratio = 2; % Required thrust to weight ratio 2:1
```

```
for design = 1:numDesigns
```

```
    for material = 1:numMaterials
```

```
        droneMass = baseMass + (4 * armMass(design,material));
```

```
        droneWeight = droneMass * g;
```

```
        maxPayload = (totalThrust_N/(2*g)) - droneMass; % Derived from TotalThrust/Weight
```

```
        totalMass=droneMass + maxPayload;
```

```
        totalWeight=totalMass*g;
```

```
        TWR = totalThrust_N/totalWeight;
```

```
        maxPayload = max(maxPayload,0); % To prevent negative payload value
```

```
        meetsPayload = maxPayload >= minimumPayload; % Holds a pass/fail value for meeting  
0.5 kg requirement
```

```
        meetsTWR = TWR >= ratio; % Holds a pass/fail value for meeting 2:1 ratio
```

```
        meetsRequirement = meetsPayload && meetsTWR; % Sees if design meets both ratio and  
payload values.
```

```
        Design{i} = designs{design};
```

```
        Material{i} = materials{material};
```

```
        DroneMass(i) = droneMass;
```

```
        PayloadCapacity(i) = maxPayload;
```

```
        ThrustToWeight(i) = TWR;
```

```
        Pass(i) = meetsRequirement;
```

```
        i = i+1;
```

```
    end
```

```
end
```

```
results = table(Design',Material',DroneMass',PayloadCapacity',ThrustToWeight',Pass',...
```

```
    'VariableNames',{'Design','Material','DroneMass_kg',...  
    'MaxPayload_kg','TWR','MeetsRequirement'}); % Creates a table to display each value and  
result
```

```
disp(results)
```

```
end
```

```
results = thrustToWeightAnalysis(baseMass,armMass,totalThrust_N,designs,materials);
```

```
%% Task 4: FEA for Two Drone Arm Designs
```

```
designNames = ["Design 1", "Design 2","Design 3"];
```

```
% Store the two STL file names in a string array
```

```
CADFiles = ["Design1.stl", "Design2.stl","Design3.stl"];
```

```
%% Show face labels for both designs
```

```
for d = 1:numel(CADFiles)
```

```
    % Create an empty 3D static structural model
```

```
    faceModel = createpde("structural", "static-solid");
```

```
    % Import the current STL geometry
```

```
    importGeometry(faceModel, CADFiles(d));
```

```
    % Convert geometry dimensions from millimeters to meters
```

```
    scale(faceModel.Geometry, 0.001);
```

```
    % Display the geometry with face numbers
```

```
    figure;
```

```
    pdegplot(faceModel, "FaceLabels", "on","FaceAlpha", 0.5);
```

```
    axis equal;
```

```
    xlabel("X");
```

```
    ylabel("Y");
```

```
    zlabel("Z");
```

```
    view(3);
```

```
    title("Face Labels: " + designNames(d));
```

```
end
```

```
% These are the face for the model where we tel MATLAB to applied force and direction.
% baseFaceIDs: face attached to drone body
% motorFaceIDs:face where motor/propeller thrust is applied
```

```
baseFaceSur = [47, 4, 4];    % [Design 1 base face, Design 2 base face]
motorFaceSur = [3, 1, 9];    % [Design 1 motor face, Design 2 motor face]
```

```
thrust_N = 1 * g;           % This is motor thrust which is the upward force produced by motor
                             % system that is 1kg thrust per motor.In this task, we only analyze one drone arm so we use one
                             % motor
```

```
motorWeight_N = PerMotor_kg* g; %This is weight of the drone which is the mass from the
motor multiply the gravity that produces downward force
```

```
netForce_N = thrust_N - motorWeight_N;%Since Thrust produces upward force and motor
weight produces downward force.Then we have net force, if the total thrust is greater than the
motor weight then the drone will accelerate upward
```

```
fprintf("Motor thrust force = %.3f N\n", thrust_N)
fprintf("Motor weight force = %.3f N\n", motorWeight_N)
fprintf("Net upward force = %.3f N\n", netForce_N)
```

```
% This is to calculate the Motor mount area for each design. Motor mount
% area is the selected CAD design face where the force is applied
% Enter motor mount diameter for each design in mm.
% Design 1 motor mount diameter = 4 mm
% Design 2 motor mount diameter = 10 mm
% Design 3 motor mount diameter = 5 mm
```

```
motorMountDiameter_mm = [4, 10, 12]; %[Design 1, Design 2, Design 3]
```

```
motorMountRadius_m = (motorMountDiameter_mm / 2) * 1e-3; %turn it into meters and divided
by /2 to turn it to radius
motorMountAreas_m2 = pi .* motorMountRadius_m.^2; %All circle area are using formula
pi(radius)^2
```

```
disp("Motor mount areas in m^2:")
```

```
%Initialize all the variables into array to use for loop.
```

```
Design = strings(0);
```

```
Material = strings(0);
```

```
MotorMountArea_m2 = [];
```

```
SurfaceTraction_N_m2 = [];
```

```
MaxDisplacement_m = []; %This means how the arms drone move or bend under load
```

```
MaxVonMisesStress_Pa = []; %This means the total combined stress value to use to calculate the FOS
```

```
YieldStrength_Pa = []; %This means the stress level of the material
```

```
FactorOfSafety = [];
```

```
PassFEA = [];
```

```
FEAStatus = strings(0);
```

```
% Arm length for each in mm
```

```
armLength_mm=[93.74, 157, 250] % [Design 1 , Design 2, Design 3]
```

```
armLength_m = armLength_mm * 1e-3;
```

```
Cost_USD_per_m = [];
```

```
CostPerArm_USD = [];
```

```
TotalArmCost_USD = [];
```

```
% Run FEA for 3 designs and 6 materials
```

```
for design=1:length(CADFiles)
```

```
    CADFile=CADFiles(design);
```

```
    designName= designNames(design);
```

```
    baseFace= baseFaceSur(design);
```

```
    motorFace=motorFaceSur(design);
```

```
    motorMountArea_m2 = motorMountAreas_m2(design);
```

```
    % Surface traction = Net force / area is the force applied over a surface
```

```
    % of the drone. This means a motor force is distributed over the motor mount
```

```
    % face.
```

```
    traction_N_m2 = netForce_N / motorMountArea_m2;
```

```
    % This is to load the direction of the surface where the force would
```

```
    % applied in X,Y or Z direction
```

```
    % Based on the upward force and downward force produces by thrust and
```

```
    % motor weight, then we would use the Z direction.
```

```
    % X Direction --> [0; 0; traction]
```

```
    % Y Direction --> [0;traction; 0]
```

```
    % Z Direction --> [0; 0; traction]
```

```
    loadDirection = [0; 0; traction_N_m2];
```

```

for m=1:length(droneArmMaterials)

    structuralModel = createpde("structural","static-solid");
    importGeometry(structuralModel, CADFile);

    scale(structuralModel.Geometry, 0.001); % All our design are in mm so scale geometry
    from mm to meters

    % This is to grab the variable from the Drone Material files
    materialName = string(droneArmMaterials(m).name);
    E = droneArmMaterials(m).E_Pa;
    nu = droneArmMaterials(m).nu;
    yieldStrength = droneArmMaterials(m).yieldStrength_Pa;
    costPerMeter = droneArmMaterials(m).cost_USD_per_m;

    structuralProperties(structuralModel, "YoungsModulus",E, "PoissonsRatio",nu);

    % This is to fix drone-body connection face so it cannot be moved
    % while analyze
    structuralBC(structuralModel, "Face",baseFace, "Constraint","fixed");

    % This is to apply the motor thrust as a distributed surface traction
    % over the motor-mount face in the Z direction.
    structuralBoundaryLoad(structuralModel, "Face",motorFace,
    "SurfaceTraction",loadDirection);

    % This is to create the mesh where Matlab divides our design drone
    % arm to many small finite element so Matlab could calculate
    % stress/strain and displacement of the drone.
    generateMesh(structuralModel);
    result = solve(structuralModel);

    % Use matlab command to calculate displacement, vonmisesstress
    maxDisplacement=max(result.Displacement.Magnitude); %Matlab calculate the total
    displacement from all three direction x,y and z direction where we have formula total
    displacement= square root(x^2+y^2+z^2)
    maxStress=max(result.VonMisesStress);

    factorOfSafety= yieldStrength/maxStress; %based on the equation from student template
    on challenges

    if factorOfSafety < 1.5

```

```

        status = "Fail";
    elseif factorOfSafety <= 5
        status = "Good Design Range";
    else
        status = "Overbuilt";
    end

    % This is to analyze how much the cost per arm design

    costPerArm = costPerMeter * armLength_m(design);
    totalArmCost = 4 * costPerArm; % This is add to see how much the cost for 4 arms

    % Store results
    Design(end+1,1) = designName;
    Material(end+1,1) = materialName;
    MotorMountArea_m2(end+1,1) = motorMountArea_m2;
    SurfaceTraction_N_m2(end+1,1) = traction_N_m2;
    MaxDisplacement_m(end+1,1) = maxDisplacement;
    MaxVonMisesStress_Pa(end+1,1) = maxStress;
    YieldStrength_Pa(end+1,1) = yieldStrength;
    FactorOfSafety(end+1,1) = factorOfSafety;
    FEAStatus(end+1,1) = status;
    CostPerArm_USD(end+1,1) = costPerArm;
    TotalArmCost_USD(end+1,1) = totalArmCost;

    end
end
%% Create FEA results table of the two drone design
feaResults = table(Design, Material, MotorMountArea_m2, SurfaceTraction_N_m2, ...
    MaxDisplacement_m, ...
    MaxVonMisesStress_Pa, ...
    YieldStrength_Pa, FactorOfSafety, FEAStatus, CostPerArm_USD);

disp(feaResults)

plotLabels = categorical(Design + " - " + Material);

figure
bar(plotLabels, FactorOfSafety)
ylabel("Factor of Safety")
title("Factor of Safety by Drone Design and Material")

```

```

yline(1.5, "--", "Minimum Required FoS = 1.5")
grid on
xtickangle(45)
figure
bar(plotLabels, MaxDisplacement_m)
ylabel("Maximum Displacement (m)")
title("Maximum Displacement by Drone Design and Material")
grid on
xtickangle(45)
figure
bar(plotLabels, MaxVonMisesStress_Pa)
ylabel("MaxVonMisesStress (Pa)")
title("MaxVonMisesStress by Drone Design and Material")
grid on
xtickangle(45)
figure
bar(plotLabels, CostPerArm_USD)
ylabel("Cost Per Arm (USD)")
title("Cost Per Arm USD by Drone Design and Material")
grid on
xtickangle(45)
figure
bar(plotLabels, TotalArmCost_USD)
ylabel("Cost of 4 Arms (USD)")
title("Cost of 4 Arms by Drone Design and Material")
grid on
xtickangle(45)
% Combine thrust-to-weight results Task #3 with FEA results Task #4
% This turn both tables columns to strings
feaResults.Design = string(feaResults.Design);
feaResults.Material = string(feaResults.Material);
results.Design = string(results.Design);
results.Material = string(results.Material);

% This combined both task together
combinedResults = join(feaResults, results, ...
    "Keys", ["Design", "Material"]);
disp("Combined Payload, TWR, FEA, Stress, and Cost Results:")
disp(combinedResults)

```

```

% Additional---Compare the best design and material based on all analysis and data Result

```



```

% These are requirements: Flight safety: TWR >= 2, Payload requirement:
% payload >= 0.5 kg, FEA safety: FoS >= 1.5, finally the best price
% For our design, we hope to find the material and design that has
% ---lower stress which improves better structural performance
% ---lower cost which is an important factor for efficiency to sale
% ---Higher payload which has a better flight performance
% ---Reasonable FoS and pass the TWR ratio.

```

```

bestResults = combinedResults( ...
    combinedResults.MaxPayload_kg >= 0.5 & ...
    combinedResults.TWR >= 2 & ...
    combinedResults.FactorOfSafety >= 1.5, :);

```

```

% This helps to sort the table from all the analysis descend here meant
% that put the higher value first while ascend meant that the lower value
% go first then higher value. For example, we hope to get the max payload
% capacity so we put descend so the highest maxpayload will sort first.
rankedResults = sortrows(bestResults, ["MaxPayload_kg", "MaxVonMisesStress_Pa",
"MaxDisplacement_m", "CostPerArm_USD", "FactorOfSafety"], ...
    ["descend", "ascend", "ascend", "ascend", "ascend"]);

```

```

bestOverall = rankedResults(1,:);

```

```

disp("Best overall design-material combination:")
disp(bestOverall)

```

```

disp("Ranked overall results:")
disp(rankedResults)

```

```

% This is to Plot one selected design/material result based on our best
% choice material

```

```

% designToPlot:
% 1 = Design 1
% 2 = Design 2
% 3 = Design 3
% materialToPlot:
% 1 = Carbon Fiber
% 2 = Aluminum
% 3 = Fiberglass
% 4 = PLA
% 5 = ABS
% 6 = Wood

```

```

% User can pick any design and material to plot based on the user
% preferences
DesignPlot=1
MaterialPlot = 1;

CADFile = CADFiles(DesignPlot);
baseFace = baseFaceSur(DesignPlot);
motorFace =motorFaceSur(DesignPlot);

motorMountArea_m2 = motorMountAreas_m2(DesignPlot);
traction_N_m2 = netForce_N / motorMountArea_m2;

loadDirection = [0; 0; traction_N_m2];

plotModel = createpde("structural","static-solid");

importGeometry(plotModel, CADFile);
scale(plotModel.Geometry, 0.001);

E = droneArmMaterials(MaterialPlot).E_Pa;
nu = droneArmMaterials(MaterialPlot).nu;

structuralProperties(plotModel, "YoungsModulus", E, "PoissonsRatio", nu); %get properties
structuralBC(plotModel, "Face", baseFace,"Constraint", "fixed"); % fix drone-body connection
structuralBoundaryLoad(plotModel,"Face", motorFace, "SurfaceTraction", loadDirection);
generateMesh(plotModel);
plotResult = solve(plotModel);

% This is to visualize displacement magnitude, the color show the
% displacement magnitude at different parts of the drone arm. For example
% the dark blue means near also no movement while red color moves the most.
% To explain this because the blue end is the fixed based to the drone body
% so it doesn't move much while red is the motor mount end where it
% experiences the applied load.
figure
pdeplot3D(plotModel, "ColorMapData", plotResult.Displacement.Magnitude)
title("Displacement Magnitude: " + designNames(DesignPlot))

% This is to visualize von Mises stress based on the color diagram where the hot color
% are having more stress while lower color are meant no stress around it.
figure
pdeplot3D(plotModel, "ColorMapData", plotResult.VonMisesStress)
title("Von Mises Stress: " + designNames(DesignPlot))

```

```
% This is to visualize x, y, and z-direction. The warmer colors indicate  
% a large displacement in the positive X, Y, Z direction while the cooler color  
% indicates smaller or negative displacement.
```

```
figure
```

```
pdeplot3D(plotModel,"ColorMapData", plotResult.Displacement.ux)  
title("X Displacement: " + designNames(DesignPlot))
```

```
figure
```

```
pdeplot3D(plotModel, "ColorMapData", plotResult.Displacement.uy)  
title("Y Displacement: " + designNames(DesignPlot))
```

```
figure
```

```
pdeplot3D(plotModel, "ColorMapData", plotResult.Displacement.uz)  
title("Z Displacement: " + designNames(DesignPlot))
```